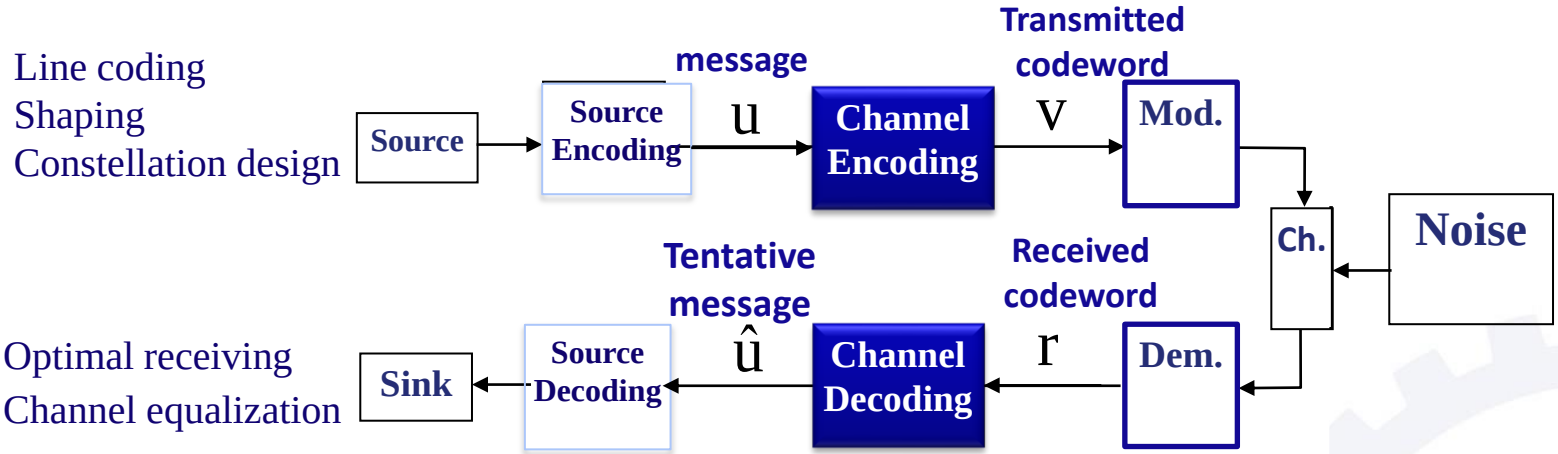
Error Control Coding

Introduction
Linear block codes
Cyclic codes
Convolutional codes

The addition of redundancy in the coded messages implies the need for **increased transmission bandwidth**. Moreover, the use of error-control coding adds ***complexity*** to the system, especially for the implementation of decoding operations in the receiver. Thus, the **design trade-offs** in the use of **error-control coding** to achieve **acceptable error performance** include considerations of **bandwidth** and **system complexity**



How to enhance reliability?

Error Control Coding (ECC)

LEARNING GUIDE

Redundancy introduced by some relation rule to detect or correct errors

The approach taken is to introduce ***redundancy*** in the channel encoder so as to reconstruct the original source sequence as accurately as possible

Theoretical basis
Conceptions
Examples of ECC

INTRODUCTION TO ECC

8.1.1 THEORETICAL BASIS

Symbol sequence

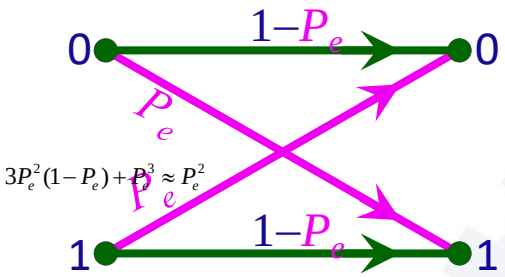
- Single symbol for error free: infinite power or zero approaching data rate

$$P_e = \frac{1}{2} erfc\left(\sqrt{\frac{E_b(1-\rho)}{2n_0}}\right)$$

- Channel description by Shannon:

  Mapping between Random symbol sequences (codewords): X→Y: P(Y/X)

  Is there a limit to data rate for error free transmission?



- Example
  - 1 is transmitted with error rate P_e
  - 1 is transmitted three times, and the majority logic decoding is used
  - Error rate is

$$3P_e^2(1-P_e)+P_e^3 \approx P_e^2$$

Trans.	Rec.	
111	111	100
	110	010
	101	001
	011	000

Error detection and correction

- Represent weathers by 3 bits
- Eight cases
  - Each one is valid
  - Error cannot be detected
- Four cases
  - Only odd errors can be detected
  - The errors cannot be corrected
- Two cases
  - Up to two errors can be detected, or one error can be corrected

- 000 (sunny)
- 001 (cloudy)
- 010 (overcast)
- 011 (rain)
- 100 (snow)
- 101 (frost)
- 110 (fog)
- 111 (hail)

- 000 (sunny)
- 011 (cloudy)
- 101 (overcast)
- 110 (rain)

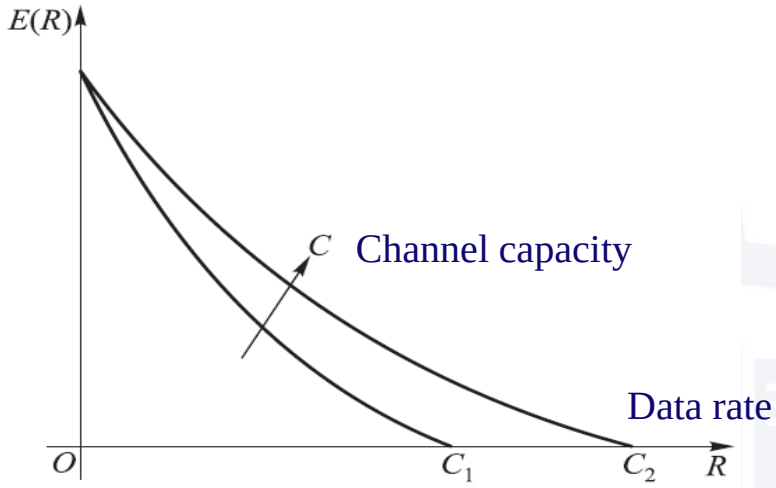
- 000 (sunny)
- 111 (rain)

Shannon Channel Coding Theorem

- **Shannon's second theorem**
  - If $R \leq C$, there ***exists*** some coding scheme, where the error rate decreases exponentially with the increase of the code length n

$$P \leq e^{-nE(R)}$$

  - If $R > C$, it is impossible to find such coding scheme.



- Good codes
  - Decoding complexity → finite length
  - Easy decoding → Algebraic structure

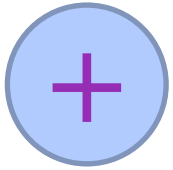
Algebraic basis for ECC

- Structural ECC by Hamming in Bell Lab in 1950, based on Galois group theory
- Finite field theory
- Binary encoding: GF(2)

elements: 0, 1

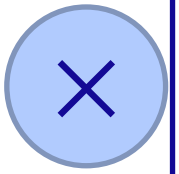
operations: +, ×

- closure



$1 + 1 = 0$
$0 + 0 = 0$
$1 + 0 = 1$
$0 + 1 = 1$

addition



$1 \times 1 = 1$
$0 \times 0 = 0$
$1 \times 0 = 0$
$0 \times 1 = 0$

multiplication

SUMMARY

Shannon’s channel coding theory: error rate decreases with the increase of coding length

ECC: obtain efficiency and reliability simultaneously

8.1.2 CONCEPTIONS

Weight and distance

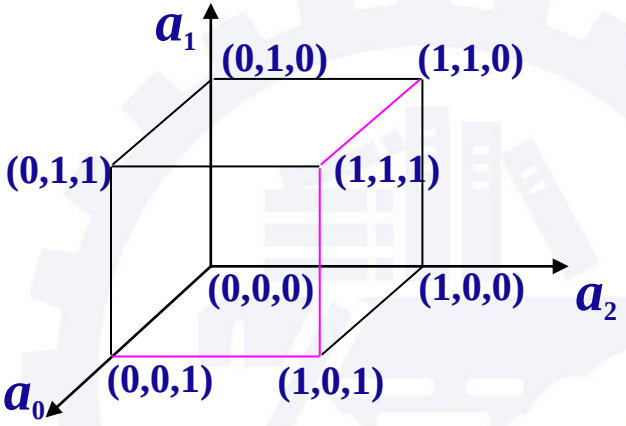
- Hamming weight W_0
  - Number of 1's in a codeword
- Hamming distance
  - Number of locations where two codewords differ
- Minimum distance d_0
  - The minimum non-zero value of all possible distances for a code
- Geometry description
  - The larger the distance, the better the reliability
  - Resemble the $d_{\min}$ wherein Euclidean distance in constellation

Target: maximize d_0

110 Weight 2

001

Distance 3 Euclidean distance $\sqrt{3}$



d_0 and the capacity of error detection and correction

- detecting up to e errors

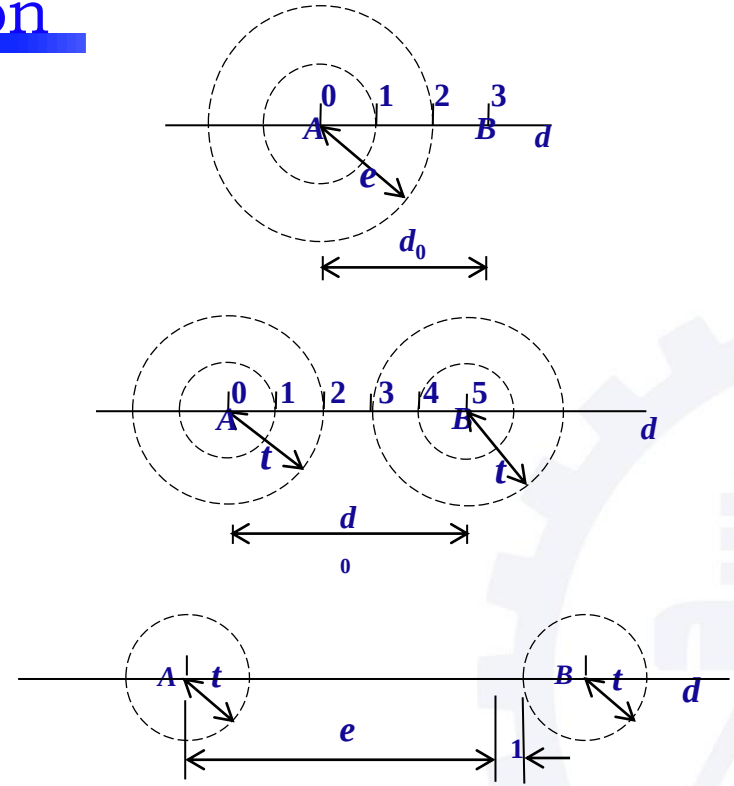
$$d_0 \geq e+1$$

- Correcting up to t errors

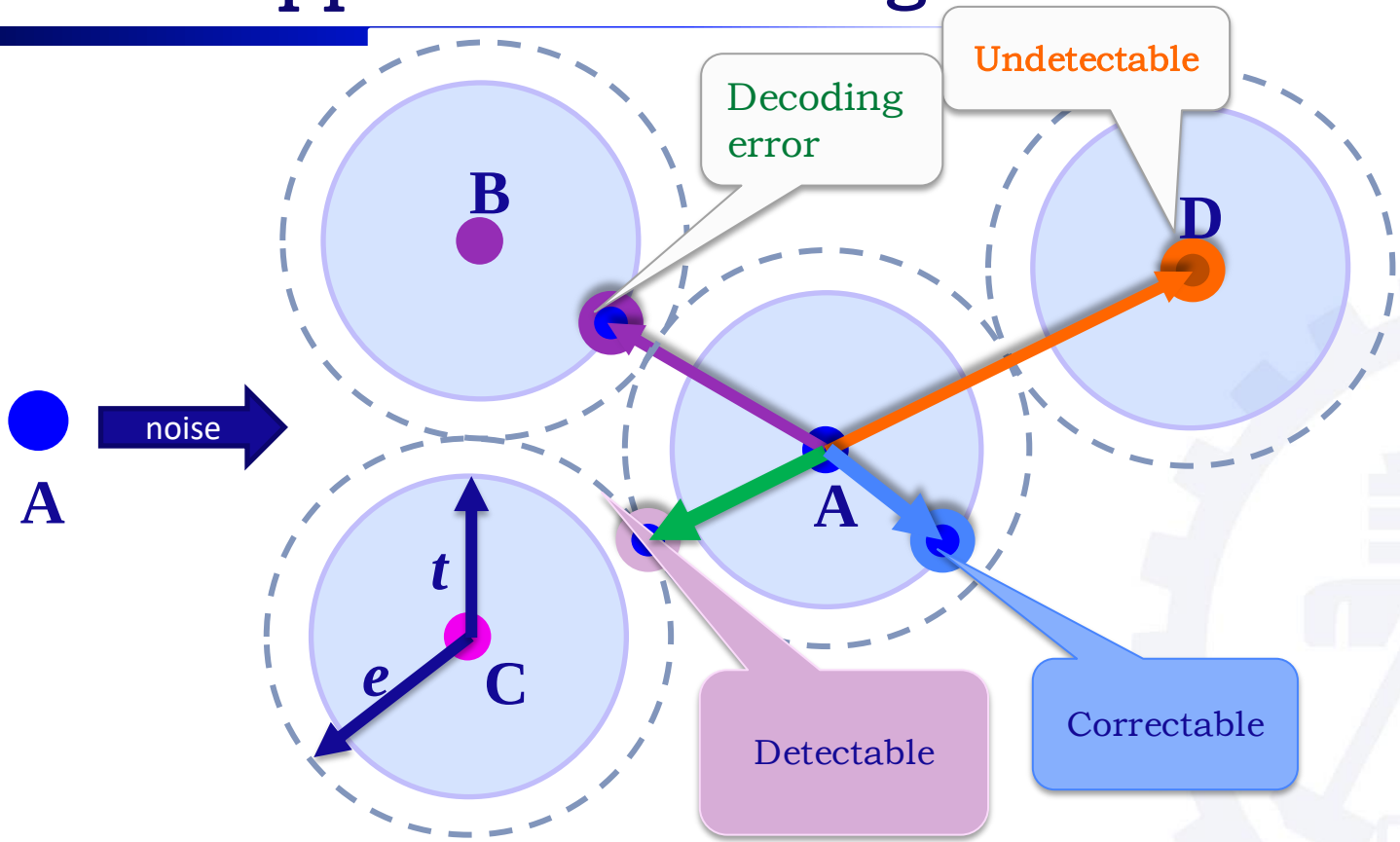
$$d_0 \geq 2t+1$$

- Correcting up to t errors and detecting up to e errors

$$d_0 \geq t+e+1, \qquad e > t$$

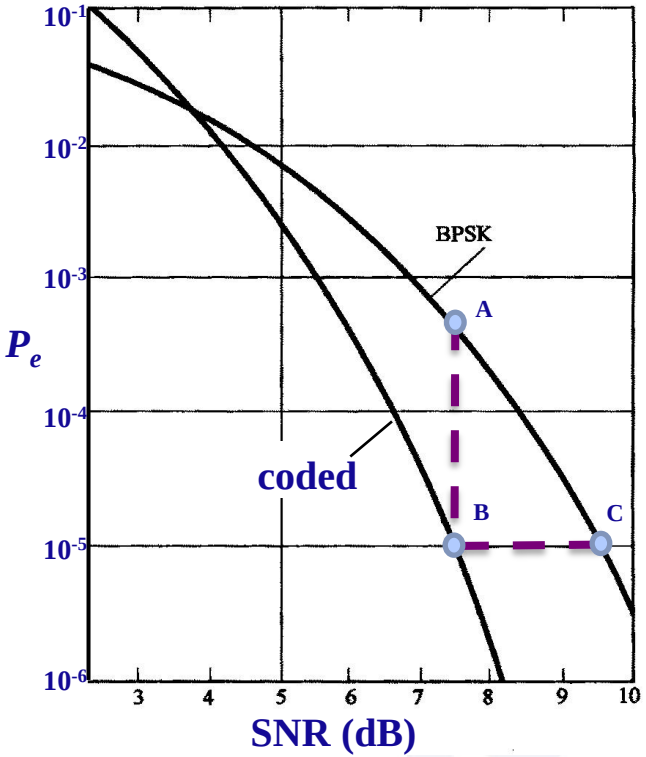


What happens in decoding



Coding gain and coding rate

- Coding gain
  - the power "saving" given by a coded scheme over an uncoded counterpart
- Coding rate k/n
  - k : dimension, n: coding length
- 2^k codewords selected from 2^n symbol sequences

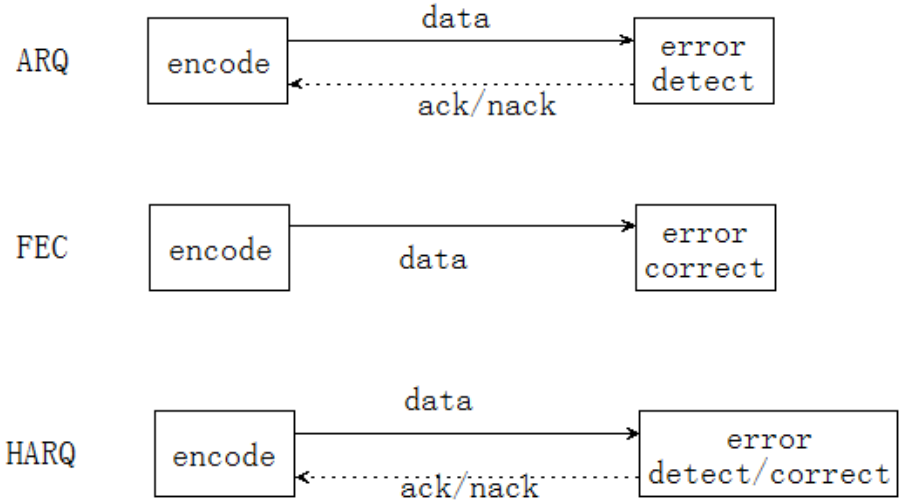


Classification of Channel Coding

- **Function：correcting and detecting**
- **Error pattern：random and burst**
- **Linearity：linear and nonlinear**
- **Information form：systematic and nonsystematic**
- **Memory length: block codes and convolutional**

ECC methods

- **Automatic Repeat reQuest, ARQ**
- **Forward Error Correction, FEC**
- **Hybrid ARQ, HARQ**



SUMMARY

Generally, coding gain increases with minimum distance, while coding rate decreases.

ECC in ARQ, FEC, and HARQ



8.1.3 EXAMPLES

Single parity check (SPC) code

- Only one redundant bit
- Detect odd number of errors

$$a_{n-1} + a_{n-2} + \cdots + a_0 = 0$$

- The redundant bit is the modulo-2 sum of all information.

奇偶校验码

- 监督位只有1位，能够检测奇数个错码
- 偶校验码：选择监督码 a_0 使码组中有偶数个“1”

$$a_{n-1} + a_{n-2} + \cdots + a_0 = 0$$

  - 接收端计算上式，“1”：错码，“0”：无错

- 奇校验码：选择监督码 a_0 使码组中有奇数个“1”

$$a_{n-1} + a_{n-2} + \cdots + a_0 = 1$$

Array code

- Horizontal SPC codes: one check bit for each row
  - $a_0^1\ a_0^2\ \ldots\ a_0^m$
  - detect odd number of errors in each row
  - Detect burst error with length up to the number of rows

$$\begin{matrix} a_{n-1}^1 & a_{n-2}^1 & \cdots & a_1^1 & a_0^1 \\ a_{n-1}^2 & a_{n-2}^2 & \cdots & a_1^2 & a_0^2 \\ \cdots & \cdots & & \cdots & \\ a_{n-1}^m & a_{n-2}^m & \cdots & a_1^m & a_0^m \end{matrix}$$

- Horizontal and vertical SPC codes：
  - Add one check bit for each column additionally
  - Detect some error patterns with even number of errors

- 水平垂直奇偶校验码：每列码组加1个监督位
  - $c_{n-1}\ c_{n-2}\ \ldots\ c_1\ c_0$为按列校验的$n$个监督位
  - 可检测某行/列的所有奇数个错误
  - 可检测不超过行/列数的突发错码
  - 可以发现部分偶数个错误
  - 对构成矩形四角的错码无法检测

Repetition codes

- (n, 1) code
- Coding rate=1/n ， $d_0=n$

- **000（sunny）**
- **111（rain）**

- Majority logic decoding
  - If the error rate in channel is p， the probability that i errors occur among the n bits is

$$P_i = \binom{n}{i} P^i (1-P)^{n-i}$$

  - For odd n, the undetectable error rate is

$$q = \sum_{i=(n+1)/2}^{n} \binom{n}{i} p^i (1-p)^{n-i}$$

Constant ratio codes

- Constant weight codes
- Then number of 1's keeps fixed.
- Easy to implement
- For transmitting letters and characters

- 优点: 简单
  - 适于传输电传机或键盘设备产生的字母和符号
  - 不适合二进制随机数字序列

Telegram in China

3:2 digital protection code

number	codeword
0	0 1 1 0 1
1	0 1 0 1 1
2	1 1 0 0 1
3	1 0 1 1 0
4	1 1 0 1 0
5	0 0 1 1 1
6	1 0 1 0 1
7	1 1 1 0 0
8	0 1 1 1 0
9	1 0 0 1 1

SUMMARY

SPC and its extension to array code

Repetition code and majority logic decoding

Constant weight codes have the same weight

Syndrome and Hamming codes
Parity check equations and matrix
Generator matrix and encoding
Syndrome-based decoding

8.2 LINEAR BLOCK CODES

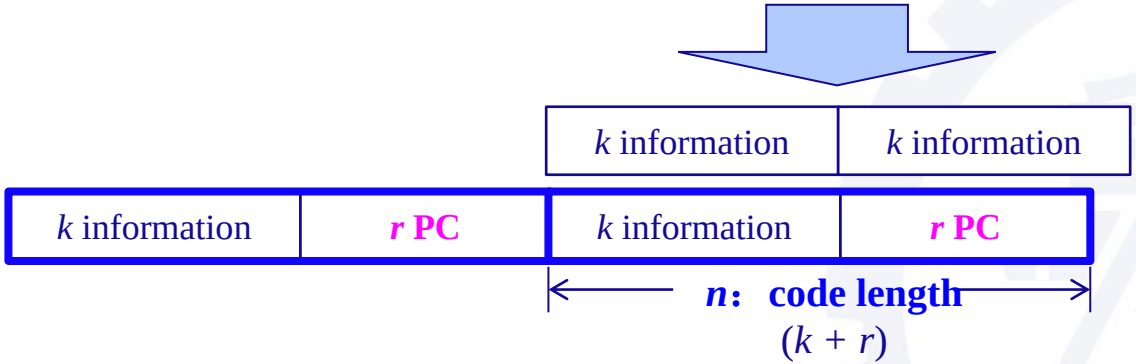
8.2.1 SYNDROME AND HAMMING CODES

block codes: information part and parity check (PC) part

- (3, 1) repetition code:
- an (n, k) block code

Repetition code: (n,1)
SPC code: (n, n–1)

information	0	1
codeword	000	111



Representation of error locations

- Increase the dimension of syndrome: r bits
- Distinguish $2^r - 1$ error patterns, one for each)
- For single error correction（t =1），

$$2^r - 1 \geq n$$
$$2^r \geq k + r + 1$$

- Hamming codes：(2^m-1, 2^m-1-m)

$t = 1 \quad d_0 = 3$

$d_0 = 3$

- 汉明码：能够纠正1位错且编码效率最高的一种线性分组码
  - $r = 2$: (3,1)码 1/3 = 33.3%
  - $r = 4$: (15,11)码 11/15 = 73.3%
  - $r = 5$: (31,26)码 26/31 = 83.9%
  - $r = 7$: (127,120)码 120/127 = 94.5%
  - $r = 10$: (1023,1013)码 1013/1023 = 99.0%

Hamming bound

- The number of syndrome is not less than the number of correctable error patterns, i.e.,

$$2^{n-k} \geq \sum_{i=0}^{t} \binom{n}{i}$$

- Perfect codes： meet Hamming bound with equality
  - Hamming codes, repetition codes, triple-error correcting (23,12) Golay binary code, double-error correcting (11,6) Golay ternary code

Example

- Determine the dimension of a double-error correcting code with code length 7.

$t = 2$

$$2^{n-k} \geq \sum_{i=0}^{2} \binom{n}{i} = \binom{n}{0} + \binom{n}{1} + \binom{n}{2}$$

$$= 1 + n + \frac{n(n-1)}{2} = 1 + \frac{n^2 + n}{2}$$

$$n - k \geq \log_2 \left(1 + \frac{n^2 + n}{2} \right)$$

$n = 7$

$$7 - k \geq \log_2 29 = 4.68$$

$$k = \lfloor 2.142 \rfloor = 2$$

Syndrome and error pattern

- A (7,4) Hamming code
- syndrome

$$S_1 = a_6 + a_5 + a_4 + a_2$$

$$S_2 = a_6 + a_5 + a_3 + a_1$$

$$S_3 = a_6 + a_4 + a_3 + a_0$$

For any codeword, the corresponding syndrome is zero.

$S_1 S_2 S_3$	Erro Loc.	$S_1 S_2 S_3$	Erro Loc.
001	a_0	101	a_4
010	a_1	110	a_5
100	a_2	111	a_6
011	a_3	**000**	**Error free**

SUMMARY

Syndrome is actually the results of parity check.

Hamming bound

Hamming codes are perfect codes with single-error correction.

8.2.2 PARITY CHECK EQUATIONS AND PARITY CHECK MATRIX

Parity check equations

equations

$$\begin{cases} a_6 + a_5 + a_4 + a_2 = 0 \\ a_6 + a_5 + a_3 + a_1 = 0 \\ a_6 + a_4 + a_3 + a_0 = 0 \end{cases}$$

then

$$\begin{cases} a_2 = a_6 + a_5 + a_4 \\ a_1 = a_6 + a_5 + a_3 \\ a_0 = a_6 + a_4 + a_3 \end{cases}$$

INF. $a_6a_5a_4a_3$	PC. $a_2a_1a_0$	INF. $a_6a_5a_4a_3$	PC. $a_2a_1a_0$
0000	000	1000	111
0001	011	1001	100
0010	101	1010	010
0011	110	1011	001
0100	110	1100	001
0101	101	1101	010
0110	011	1110	100
0111	000	1111	111

(7,4)Hamming code

Parity check matrix

$$\begin{cases} a_6 + a_5 + a_4 + a_2 = 0 \\ a_6 + a_5 + a_3 + a_1 = 0 \\ a_6 + a_4 + a_3 + a_0 = 0 \end{cases}$$

- (7,4) Hamming code
- Equations to matrix
  - $\mathbf{H}\mathbf{A}^T = \mathbf{0}^T$
  - $\mathbf{A}\mathbf{H}^T = \mathbf{0}$

$$\begin{cases} 1 \cdot a_6 + 1 \cdot a_5 + 1 \cdot a_4 + 0 \cdot a_3 + 1 \cdot a_2 + 0 \cdot a_1 + 0 \cdot a_0 = 0 \\ 1 \cdot a_6 + 1 \cdot a_5 + 0 \cdot a_4 + 1 \cdot a_3 + 0 \cdot a_2 + 1 \cdot a_1 + 0 \cdot a_0 = 0 \\ 1 \cdot a_6 + 0 \cdot a_5 + 1 \cdot a_4 + 1 \cdot a_3 + 0 \cdot a_2 + 0 \cdot a_1 + 1 \cdot a_0 = 0 \end{cases}$$

- A is a codeword if and only if $\mathbf{A}\mathbf{H}^T = \mathbf{0}$

$$\underbrace{\begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}}_{\mathbf{H}} \underbrace{\begin{bmatrix} a_6 \\ a_5 \\ a_4 \\ a_3 \\ a_2 \\ a_1 \\ a_0 \end{bmatrix}}_{\mathbf{A}^T} = \underbrace{\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}}_{\mathbf{0}^T}$$

$$\mathbf{A} = \begin{bmatrix} a_6 & a_5 & a_4 & a_3 & a_2 & a_1 & a_0 \end{bmatrix}$$

Properties of H

- Linearly independent rows with full rank
- *Actually, you can get a linearly dependent H matrix easily by adding some redundant rows*

$$\mathbf{H}=\left[\begin{array}{cccc:ccc}1&1&1&0&1&0&0\\1&1&0&1&0&1&0\\1&0&1&1&0&0&1\end{array}\right]=\left[\mathbf{P}_{r\times k},\mathbf{I}_{\mathbf{r}}\right]$$

- Systematic H matrix
  - With elementary row transformation (sometimes column permutation is necessary), nonsystematic H matrix can be converted to systematic form. Essentially, that is Gaussian elir

- 各行线性无关
  - 每一行：一个监督关系
  - 列数：码长 n
  - 行数：监督位的数目 r
  - 不应含有全零列
  - 不应含有重复列

$$\mathbf{H}=\left[\begin{array}{cccc:ccc}1&1&1&0&1&0&0\\1&1&0&1&0&1&0\\1&0&1&1&0&0&1\end{array}\right]$$

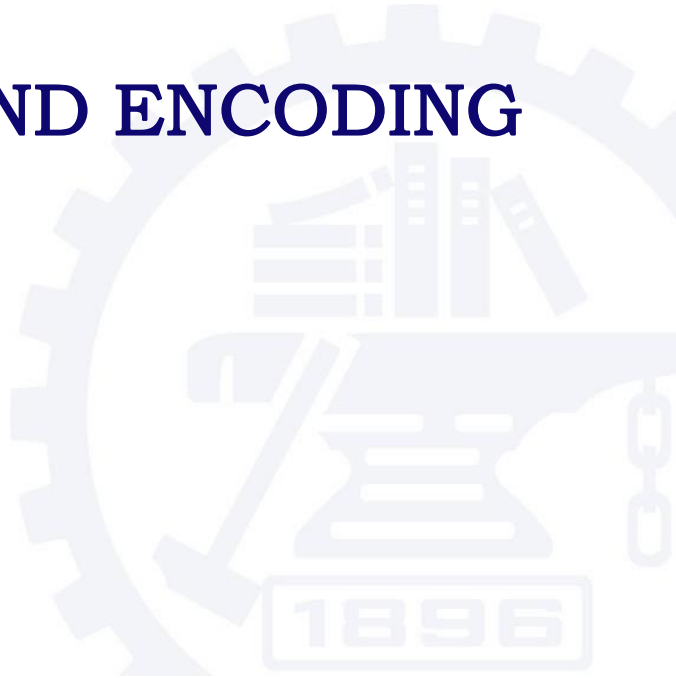
- 系统（典型）形式校验矩阵 $\mathbf{H}=[\mathbf{P},\mathbf{I}_r]$
  - 对应系统码形式的码字
    - 信息位序列出现在码字的固定位
  - 非系统形式转为系统形式：初等行变换

发端如何产生一个完整码字?
类似的矩阵形式?

SUMMARY

Using parity check equations to get redundant bits
H matrix consists of the coefficients of equations
Systematic H matrix corresponds to systematic codeword

8.2.3 GENERATOR MATRIX AND ENCODING



Generator matrix

- Observe parity check equations
- Redundant bits: The linear combinations of information bits
- *Information bits appear unchanged*

$$\begin{cases} a_6 + a_5 + a_4 + a_2 = 0 \\ a_6 + a_5 + a_3 + a_1 = 0 \\ a_6 + a_4 + a_3 + a_0 = 0 \end{cases}$$

$$\begin{bmatrix} a_2 \\ a_1 \\ a_0 \end{bmatrix} = \mathbf{P} \begin{bmatrix} a_6 \\ a_5 \\ a_4 \\ a_3 \end{bmatrix}$$

$$\mathbf{Q} = \mathbf{P}^{\mathrm{T}}$$

$$\mathbf{G} = [\mathbf{I_k}\mathbf{Q}] = \left[\begin{array}{cccc:ccc} 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{array}\right]$$

- **G** is full rank with a codeword being each row.

$$\mathbf{A} = [a_6 \quad a_5 \quad a_4 \quad a_3 \quad a_2 \quad a_1 \quad a_0]$$

From G, we get the linear block codes…

- closure: the sum of any two codewords is still a codeword

proof: if A1 and A2 are two codewords, we have

$$A_1 \cdot H^T = 0, \qquad A_2 \cdot H^T = 0$$

$$A_1 \cdot H^T + A_2 \cdot H^T = (A_1 + A_2) H^T = 0$$

$\Rightarrow (A_1 + A_2)$ is still a codeword

- All zero word is a codeword
- Minimum distance is the minimum weight of nonzero codeword

Proof: d(A1,A2)=d(A1+A2,0)=d(A3,0)=wt(A3)
min d(A1,A2)=min wt(A3)

Example: encoding

- n=7, k=4, r=3
- H matrix by equations

$$\begin{cases} a_2 = a_6 + a_5 + a_4 \\ a_1 = a_6 + a_5 + a_3 \\ a_0 = a_6 + a_4 + a_3 \end{cases} \qquad \mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{Q} = \mathbf{P}^{\mathrm{T}}$$

- Generator matrix G

$$\mathbf{G} = [\mathbf{I}_k \mathbf{Q}] = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

- encoding: $\mathbf{A} = [a_6 \quad a_5 \quad a_4 \quad a_3]\mathbf{G}$

Inf. $a_6 a_5 a_4 a_3$	PC. $a_2 a_1 a_0$	Inf. $a_6 a_5 a_4 a_3$	PC. $a_2 a_1 a_0$
0000	000	1000	111
0001	011	1001	100
0010	101	1010	010
0011	110	1011	001
0100	110	1100	001
0101	101	1101	010
0110	011	1110	100
0111	000	1111	111

Duality and dual codes

- **H** matrix and **G** matrix are dual mutually.

$$\mathbf{H} = [\mathbf{P} \mid \mathbf{I}_{n-k}] \qquad \mathbf{G} = [\mathbf{I}_k \mid \mathbf{Q}]$$

$$\mathbf{P} = \mathbf{Q}^T \qquad \mathbf{Q} = \mathbf{P}^T$$

$$\mathbf{H}\mathbf{G}^T = \mathbf{0} \qquad \mathbf{G}\mathbf{H}^T = \mathbf{0}$$

- (n,k) code is the dual code of (n,n-k) code, vice versa.
- The dual of a (7,4) Hamming code is a (7,3) code.

$$\mathbf{G}(n, n-k) = \mathbf{H}(n, k)$$

$$\mathbf{H}(n, n-k) = \mathbf{G}(n, k)$$

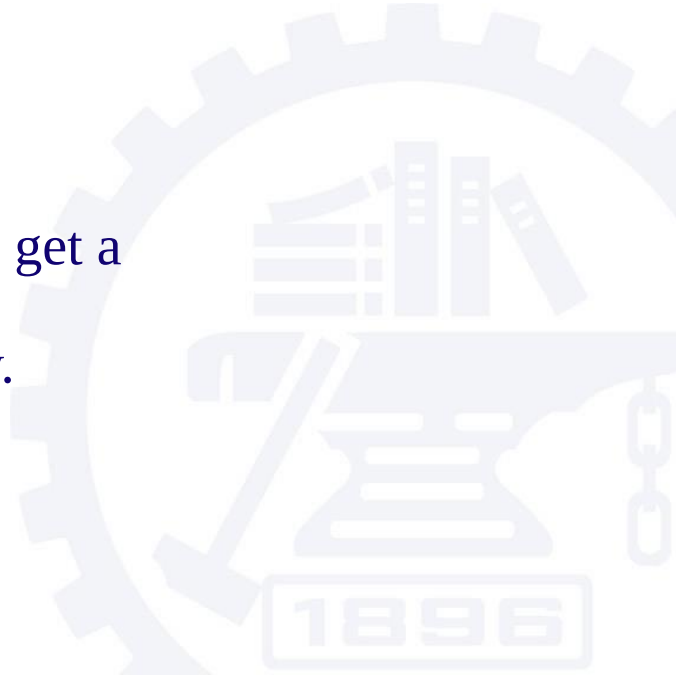
SUMMARY

Multiply information sequence by **G** matrix to get a codeword

G is full rank with a codeword being each row.

The linear block code is closed.

G matrix and **H** matrix are dual each other.



8.2.4 SYNDROME-BASED DECODING

Syndrome-based

- error pattern

$$\mathbf{E} = \mathbf{A} + \mathbf{B}$$

- Error free if $\mathbf{E} = 0$

$$e_i = \begin{cases} 0, & \text{error} \\ 1, & \text{Error free} \end{cases}$$

- Syndrome

$$\mathbf{S} = \mathbf{B}\mathbf{H}^{\mathrm{T}} = (\mathbf{A} + \mathbf{E})\mathbf{H}^{\mathrm{T}} = \mathbf{A}\mathbf{H}^{\mathrm{T}} + \mathbf{E}\mathbf{H}^{\mathrm{T}}$$

$\mathbf{A}\mathbf{H}^{\mathrm{T}} = 0$

- S is determined by error pattern E

$$\mathbf{A} = \mathbf{B} + \mathbf{E}$$

$A = 0110011$

$B = 1110011$

$E = 1000000$

$$\mathbf{A} = \begin{bmatrix} a_{n-1} & a_{n-2} & \cdots & a_1 & a_0 \end{bmatrix} \quad \text{Trans.}$$

$$+\ \mathbf{B} = \begin{bmatrix} b_{n-1} & b_{n-2} & \cdots & b_1 & b_0 \end{bmatrix} \quad \text{Received}$$

$$\mathbf{E} = \begin{bmatrix} e_{n-1} & e_{n-2} & \cdots & e_1 & e_0 \end{bmatrix}$$

Inf. $a_6 a_5 a_4 a_3$	PC. $a_2 a_1 a_0$	Inf. $a_6 a_5 a_4 a_3$	PC. $a_2 a_1 a_0$
0000	000		111
0100	110		001
0101	101		010
0110	011	1110	100
0111	000	1111	111

$$\mathbf{S}^{\mathrm{T}} = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \begin{matrix} 0 \\ 0 \\ 1 \end{matrix}$$

Hard decision vs. Soft decision decoding

- **Hard decision decoding**
  - Two-level quantification
  - Hamming distance
  - Easy to implement with integer operations, but poor performance

- **Soft decision decoding**
  - More than two levels quantification
  - Euclidean distance
  - Complicated with real number operations, but better performance

Minimum Hamming distance is the maximal likelihood (ML)

- Maximal a *posterior*, MAP

$$P(\mathbf{E}) = \sum_{\mathbf{r}} P(\mathbf{E}|\mathbf{r})P(\mathbf{r})$$

$$\hat{\mathbf{v}} = \arg\max_{\mathbf{v}} P(\mathbf{v}|\mathbf{r}) = \arg\max_{\mathbf{v}} P(\mathbf{r}|\mathbf{v}) = \arg\min_{\mathbf{v}} d_0(\mathbf{v},\mathbf{r})$$

- With equal probability, MAP is identical to ML

$$P(\mathbf{r} \mid \mathbf{v}) = p_e^{d(\mathbf{v},\mathbf{r})} \left(1 - p_e\right)^{n - d(\mathbf{v},\mathbf{r})}$$

$$= \left(1 - p_e\right)^n \left(\frac{p_e}{1 - p_e}\right)^{d(\mathbf{v},\mathbf{r})}$$

$$d(\mathbf{v},\mathbf{r}) = d_0(\mathbf{v},\mathbf{r})$$

SUMMARY

Syndrome corresponds to error pattern.

Syndrome-based decoding actually ML decoding.

Soft decision decoding performs better than hard decision decoding, while with large complexity.

Code polynomial and generator polynomial
Generator matrix and parity check polynomial
Encoding in systematic/nonsystematic form
Error detection and error correction

8.3 CYCLIC CODES

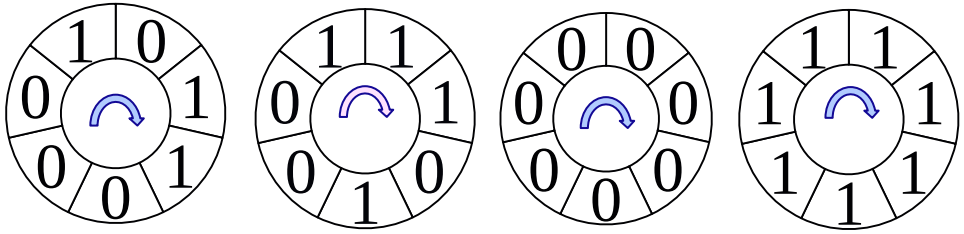
8.3.1CODE POLYNOMIAL AND GENERATOR POLYNOMIAL

Cyclic version of Hamming code

- A (7, 4) Hamming code

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

- A codeword with cyclic shift is still a codeword.
- All possible codewords below



Inf. $a_6\,a_5\,a_4\,a_3$	PC. $a_2\,a_1\,a_0$	Inf. $a_6\,a_5\,a_4\,a_3$	PC. $a_2\,a_1\,a_0$
0000	000	1000	101
0001	011	1001	110
0010	110	1010	011
0011	101	1011	000
0100	111	1100	010
0101	100	1101	001
0110	001	1110	100
0111	010	1111	111

Code polynomial

- The power stands for the bit location, and the coefficient for the value of bit.
- A polynomial defined over GF(2) with the degree no more than *n*-1

$$\mathbf{V} = (a_{n-1}, a_{n-2}, \cdots, a_1, a_0) \qquad \mathbf{V} = 1100010 \quad v(x) = x^6 + x^5 + x$$

$$v(x) = a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \cdots + a_1 x + a_0$$

- If we shift every component of ***v*** cyclically one place to left, we obtain the following code polynomial:

$$v^{(1)}(x) = a_{n-2}x^{n-1} + a_{n-3}x^{n-2} + \cdots + a_0 x + a_{n-1}$$

$$v^{(1)}(x) = xv(x) \bmod (x^n + 1)$$

- Similarly, cyclically shift every component of v k places to left:

$$v^{(k)}(x) = x^k v(x) \bmod (x^n + 1)$$

$$\mathbf{V}^{(1)} = 1000101$$

$$v^{(1)}(x) = xv(x) \bmod \left(x^7 + 1\right)$$

$$\equiv \frac{x^7 + x^6 + x^2}{x^7 + 1} \qquad = x^6 + x^2 + 1$$

$$\begin{array}{r} 1 \\ x^7+1 \overline{)\, x^7 + x^6 + x^2} \\ \underline{x^7 + 1} \\ x^6 + x^2 + 1 \end{array}$$

Generator polynomial $g(x)$

- Information polynomial of degree k-1 or less

$$u(x) = u_{k-1}x^{k-1} + u_{k-2}x^{k-2} + \cdots + u_1 x + u_0 \qquad \mathbf{U} = (u_{k-1}, u_{k-2} \cdots, u_1, u_0)$$

- $\deg(v(x)) \in [n-k,\ n-1]$
- Only one code polynomial $g(x)$ who has the least degree $n-k=r$

$$g(x) = x^{n-k} + \overbrace{g_{n-k-1}x^{n-k-1} + \cdots + g_2 x^2 + g_1 x}^{r \text{ terms}} + 1$$

- Every code polynomial is a multiple of $g(x)$.

$$v(x) \equiv 0 \bmod g(x)$$

$$X^k \mathbf{g}(X) = (X^n + 1) + \mathbf{g}^{(k)}(X)$$

$$\mathbf{g}^{(k)}(X) = \mathbf{a}(X)\mathbf{g}(X)$$

$$(X^k + \mathbf{a}(X)) \cdot \mathbf{g}(X) = X^n + 1$$

$$\mathbf{g}(X) | (X^n + 1)$$

Determine g(x)

$\mathbf{g}(X)|(X^n+1)$

$$g(x) = x^{n-k} + g_{n-k-1}x^{n-k-1} + \cdots + g_2x^2 + g_1x + 1$$

- Factorize $x^n + 1$, and then get $g(x)$

$$x^7 + 1 = (x+1)(x^3 + x^2 + 1)(x^3 + x + 1)$$

- We can obtain two (7,3) cyclic codes with

$$g(x) = x^4 + x^2 + x + 1$$

or

$$g(x) = x^4 + x^3 + x^2 + 1$$

Problem

How many cyclic codes with code length 7 are there?

SUMMARY

Cyclic shift of a cyclic code can be realized by the modulo operation.

A cyclic code is completely specified by its generator polynomial.

8.3.2 GENERATOR MATRIX AND PARITY CHECK POLYNOMIAL



$g(x)$ and G

- An (n,k) cyclic code
- Generator polynomial $g(x)$→ $\mathbf{G}(x) = \begin{bmatrix} x^{k-1}g(x) \\ x^{k-2}g(x) \\ \vdots \\ xg(x) \\ g(x) \end{bmatrix}$
- **Each row consists of the coefficients, in descending order of power.**

$$\mathbf{G} = \begin{bmatrix} g_{n-k} & g_{n-k-1} & \cdots & g_0 & \cdots & \cdots & 0 \\ 0 & g_{n-k} & g_{n-k-1} & \cdots & g_0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & g_{n-k} & g_{n-k-1} & \cdots & g_0 \end{bmatrix}$$

This is in nonsystematic form.

Example A (7,4) cyclic code

$$g(x) = x^3 + x + 1$$

$$\boldsymbol{G}(x) = \begin{bmatrix} x^3 g(x) \\ x^2 g(x) \\ x g(x) \\ g(x) \end{bmatrix} = \begin{bmatrix} x^6 + x^4 + x^3 \\ x^5 + x^3 + x^2 \\ x^4 + x^2 + x \\ x^3 + x + 1 \end{bmatrix}$$

$$\boldsymbol{G} = \begin{bmatrix} 1 & 0 & 1 & 1 & & & \\ & 1 & 0 & 1 & 1 & & \\ & & 1 & 0 & 1 & 1 & \\ & & & 1 & 0 & 1 & 1 \end{bmatrix}$$

Parity check polynomial $h(x)$

- $g(x) \mid x^n + 1$ $\qquad x^n + 1 = g(x)h(x)$
- $h(x)$ is a polynomial of degree k.

$$h(x) = h_k x^k + h_{k-1} x^{k-1} + \cdots + h_1 x + h_0$$

$$x^7 + 1 = (x+1)(x^3 + x^2 + 1)(x^3 + x + 1)$$

$$g(x) = x^3 + x + 1 \qquad h(x) = x^4 + x^2 + x + 1$$

- Using $h(x)$ as generator polynomial , we can get an $(n, n-k)$ cyclic code, the dual code of the (n,k) cyclic code.

$h(x)$ and parity check matrix

- Parity check polynomial

$$h(x) = h_k x^k + h_{k-1} x^{k-1} + \cdots + h_1 x + h_0$$

- Each row of parity check matrix consists of coefficients in ascending order of power

$$H(x) = \begin{bmatrix} x^{n-k-1}h(x) \\ \vdots \\ xh(x) \\ h(x) \end{bmatrix} \quad \mathbf{H} = \begin{bmatrix} h_0 & h_1 & \cdots & h_k & \cdots & \cdots 0 \\ 0 & h_0 & h_1 & \cdots & h_k & \cdots 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots \vdots \\ 0 & 0 & \cdots & h_0 & h_1 & \cdots h_k \end{bmatrix}$$

$$\mathbf{HG^T = 0}$$

Example: A (7,4) cyclic code

$$x^7+1=(x+1)(x^3+x^2+1)(x^3+x+1)$$

$$g(x)=x^3+x+1 \qquad h(x)=1+x+x^2+x^4$$

$$\mathbf{G}=\begin{bmatrix}1&0&1&1&&&\\&1&0&1&1&&\\&&1&0&1&1&\\&&&1&0&1&1\end{bmatrix} \qquad \mathbf{H}=\begin{bmatrix}1&1&1&0&1&&\\&1&1&1&0&1&\\&&1&1&1&0&1\end{bmatrix}$$

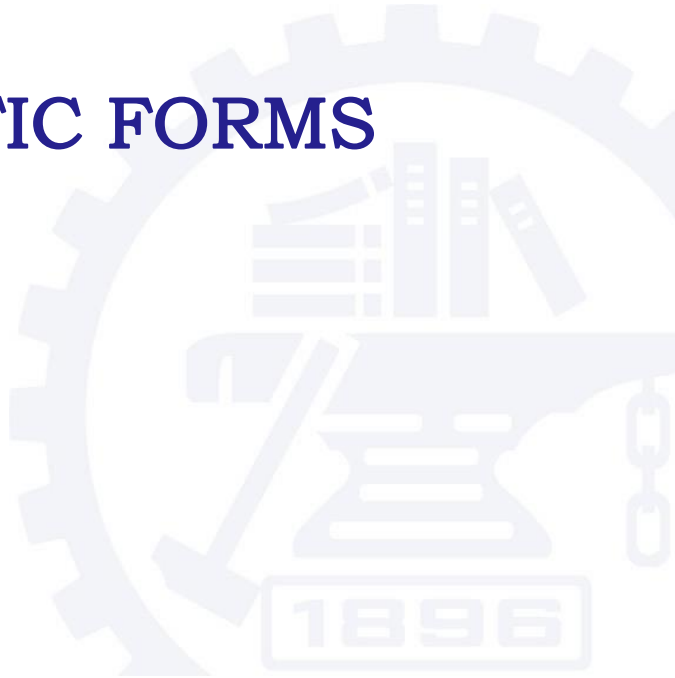
$$\mathbf{G}_{\text{sys}}=\begin{bmatrix}1&0&0&0&1&0&1\\0&1&0&0&1&1&1\\0&0&1&0&1&1&0\\0&0&0&1&0&1&1\end{bmatrix} \qquad \mathbf{H}_{\text{sys}}=\begin{bmatrix}1&1&1&0&1&0&0\\0&1&1&1&0&1&0\\1&1&0&1&0&0&1\end{bmatrix}$$

SUMMARY

Generator matrix can be constructed from generator polynomial and its cyclic shift versions.

So can parity check matrix.

8.3.3 ENCODING IN NONSYSTEMATIC/SYSTEMATIC FORMS



Nonsystematic form

- Multiply information polynomial u(x) by generator polynomial g(x)

$$v(x)=u(x)g(x)$$

V=UG

e.g., $g(x) = x^3 + x + 1$ ， what is the codeword for information (1101) ？

Systematic form

- Information polynomial

$$u(x) = u_{k-1}x^{k-1} + u_{k-2}x^{k-2} + \ldots + u_1 x + u_0$$

- Multiply u(x) by x^{n-k} to set aside n-k places for redundant bits

$$x^{n-k}u(x) = u_{k-1}x^{n-1} + u_{k-2}x^{n-2} + \cdots + u_1 x^{n-k+1} + u_0 x^{n-k}$$

- Divided $x^{n-k}u(x)$ by g(x), and get the remainder $b(x)$

$$x^{n-k}u(x) = \underbrace{q(x)}_{\text{quotient}}g(x) + \underbrace{b(x)}_{\textbf{remainder}}$$

- Append remainder $b(x)$ to $x^{n-k}u(x)$

$$v(x) = x^{n-k}u(x) + b(x) = q(x)g(x)$$

Example

$g(x) = x^3 + x + 1$， give the codeword for information 1101 in systematic form.

solution: $u(x) = x^3 + x^2 + 1$,

k-1=3, n-k=deg($g(x)$)=3, so n=k+3=7

$$\frac{x^{n-k}u(x)}{g(x)} = \frac{x^6 + x^5 + x^3}{x^3 + x + 1}$$

$$= (x^3 + x^2 + x + 1) + \frac{1}{x^3 + x + 1}$$

Code polynomial

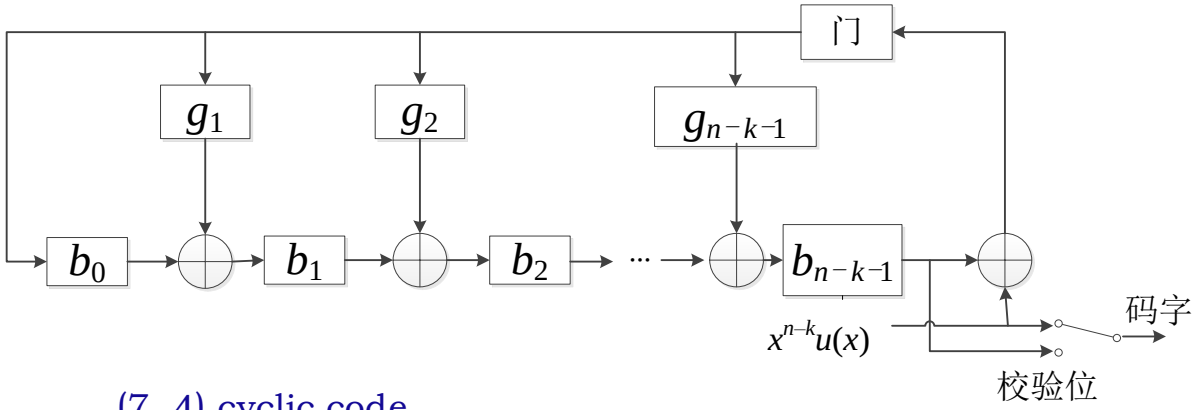
$v(x) = x^{n-k}\,u(x) + b(x) = x^6 + x^5 + x^3 + 1$

V = 1101000

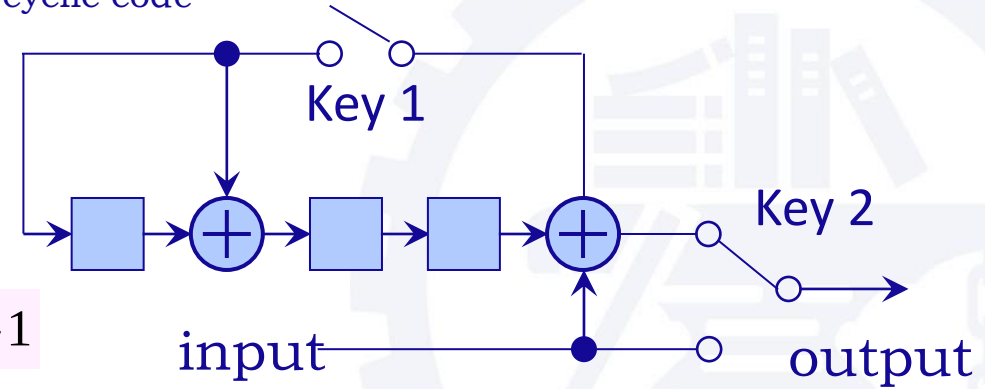
Inf. $a_6a_5a_4a_3$	PC. $a_2a_1a_0$	Inf. $a_6a_5a_4a_3$	PC. $a_2a_1a_0$
0000	000	1000	101
0001	011	1001	110
0010	110	1010	011
0011	101	1011	000
0100	111	1100	010
0101	100	1101	001
0110	001	1110	100
0111	010	1111	111

Encoding circuit

- An $(n-k)$-stage shift-register with feedback connections based on the coefficients of the generator polynomial



(7, 4) cyclic code



$$g(x) = x^3 + x + 1$$

SUMMARY

$v(x)=u(x)g(x)$

$v(x)=x^{n-k}u(x)+x^{n-k}u(x) \bmod g(x)$

8.3.4 ERROR DETECTION AND ERROR CORRECTION



Error detection

$g(x)|v(x)$

- Received polynomial $r(x)$
- Calculate syndrome polynomial $s(x)$

$$s(x) = r(x) \bmod g(x)$$

  - if $s(x) = 0$ consider no errors
  - if $s(x) \neq 0$ error occurs

- Undetectable errors： $r(x)$ is not equal to $v(x)$, but can be divisible by $g(x)$.

$$g(x) = x^4 + x^2 + x + 1$$

$$v(x) = x^6 + x^5 + x^2 + 1 \quad e(x)=x\text{^}2+x$$

$$r(x) = x^6 + x^5 + x + 1$$

$$\frac{x^6 + x^5 + x + 1}{x^4 + x^2 + x + 1} = x^2 + x + 1 + \frac{x^2 + x}{x^4 + x^2 + x + 1}$$

$$s(x) = r(x) \bmod g(x) = x^2 + x$$ Error detected!

Error correction

- Determine $e(x)$ via $s(x)$
- Tentative $v(x) = r(x) + e(x)$
- (7, 3) cyclic code

$$g(x) = x^4 + x^2 + x + 1$$

$$v(x) = x^6 + x^5 + x^2 + 1$$

$$r(x) = x^5 + x^2 + 1$$

$$e(x) = x^6$$

$$s(x) = x^5 + x^2 + 1 \bmod x^4 + x^2 + x + 1$$

$$= x^3 + x + 1$$

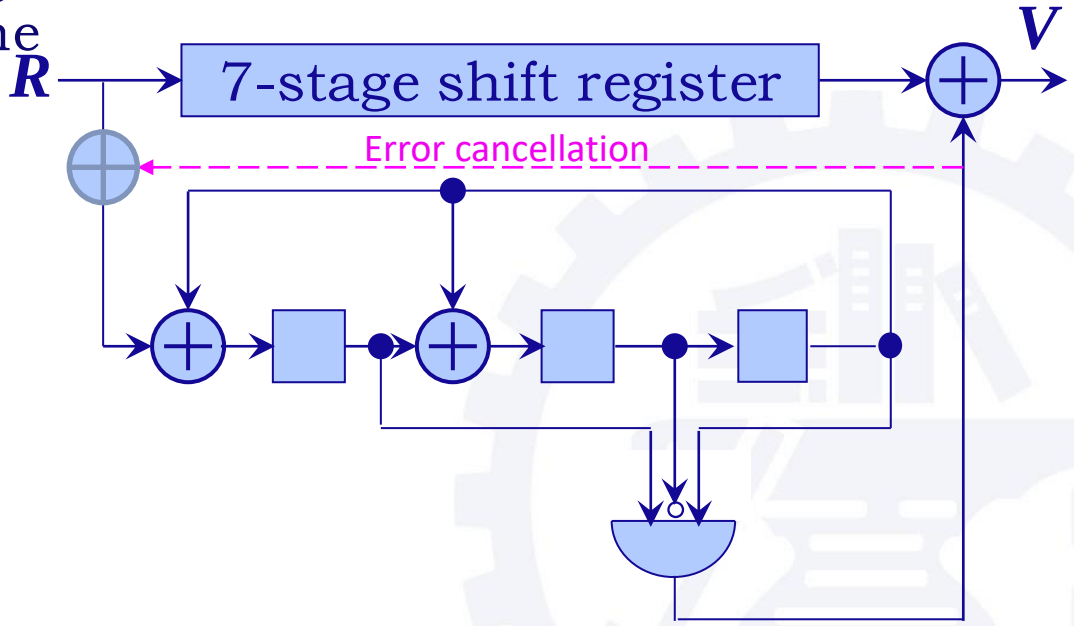
Meggitt decoder for (7,4) cyclic code

$$g(x) = x^3 + x + 1$$

- Only one-unit is needed to correct seven possible singe-error patterns in light of the cyclic character.

$$v(x) = x^6 + x^5 + x^3 + 1$$

$$r(x) = x^5 + x^3 + 1$$



$$s(x) = x^5 + x^3 + 1 \bmod x^3 + x + 1$$

$$= x^2 + 1$$

SUMMARY

Syndrome polynomial plays important role in error detection and correction.

The cyclic character can help simplify the structure of encoder and decoder

Basic structure
Viterbi algorithm

8.4 CONVOLUTIONAL CODES

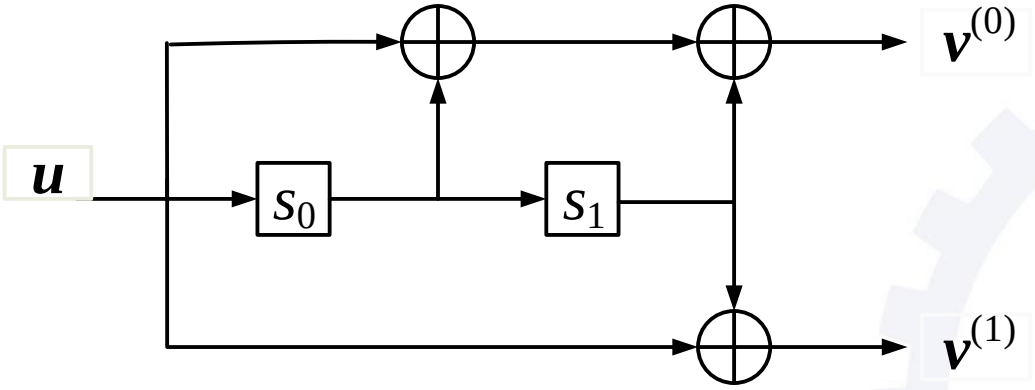
8.4.1 BASIC STRUCTURE

What is convolutional codes

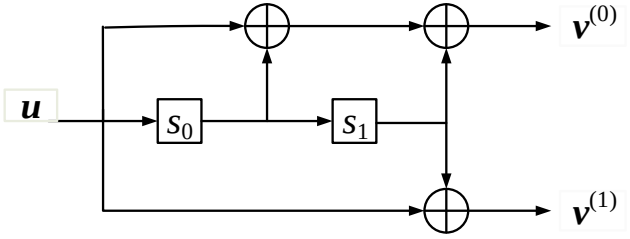
- A convolutional code is an error correcting code that processes information *serially* .
- A convolutional encoder has *memory.*
- The encoder is a *sequential circuit* or a finite-state machine.
- The *state* of the encoder is defined as the contents of the memory.

encoder

- An encoder of a memory-2 rate-1/2 convolutional encoder.



The list of encoding



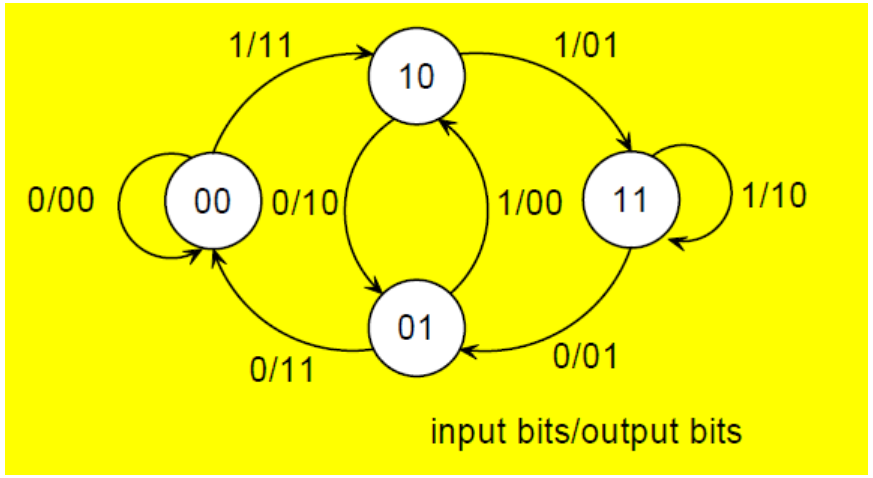
- Inputs, state transitions, and outputs

Initial state $s_0[i]s_1[i]$	Information $u[i]$	Final state $s_0[i+1]s_1[i+1]$	Outputs $v^{(0)}[i]v^{(1)}[i]$
00	0	00	00
00	1	10	11
01	0	00	11
01	1	10	00
10	0	01	10
10	1	11	01
11	0	01	01
11	1	11	10

State diagram

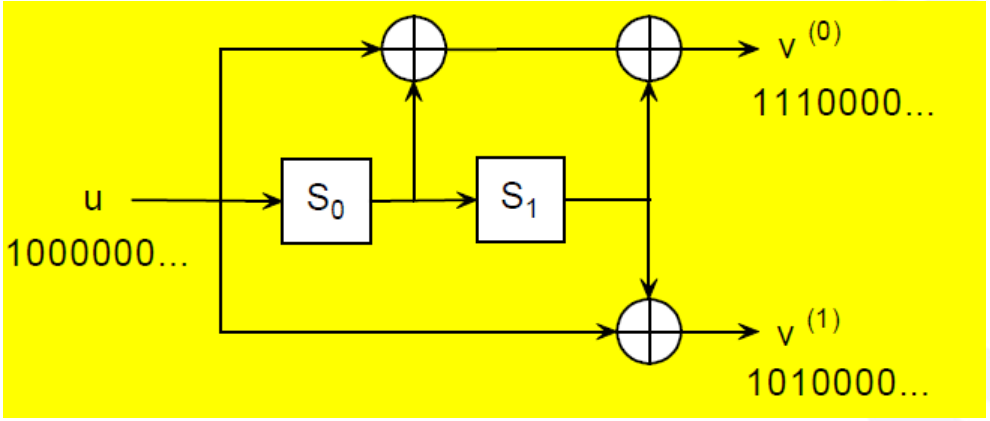
- Finite state machine (FSM)

Initial state $s_0[i]s_1[i]$	Information $u[i]$	Final state $s_0[i+1]s_1[i+1]$	Outputs $v^{(0)}[i]v^{(1)}[i]$
00	0	00	00
00	1	10	11
01	0	00	11
01	1	10	00
10	0	01	10
10	1	11	01
11	0	01	01
11	1	11	10



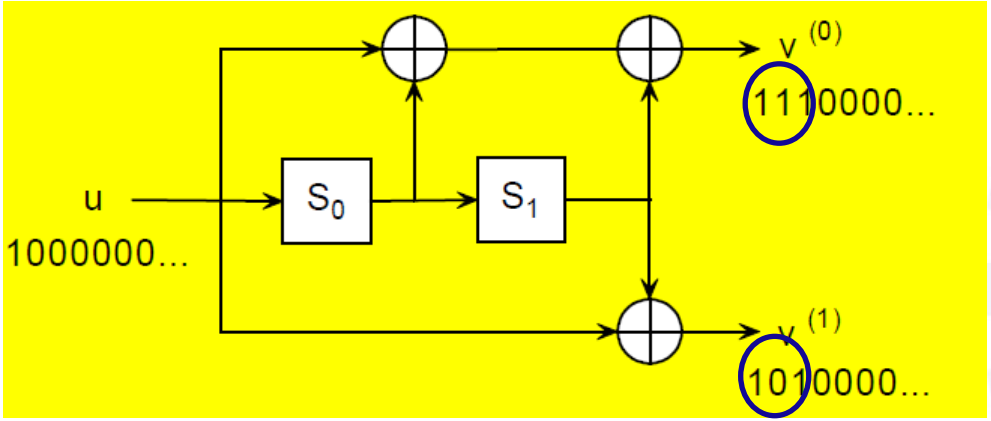
Constraint length

- The *constraint length*, is defined as the number of inputs ($u[i], u[i-1], \ldots, u[i-m]$) that affect the outputs ($v^{(0)}[i], \ldots, v^{(n-1)}[i]$) at time i.
- An (n, k, N) convolutional code, where n output bits are produced for every k input bits, has constraint length N.
- (2, 1, 3) convolutional code



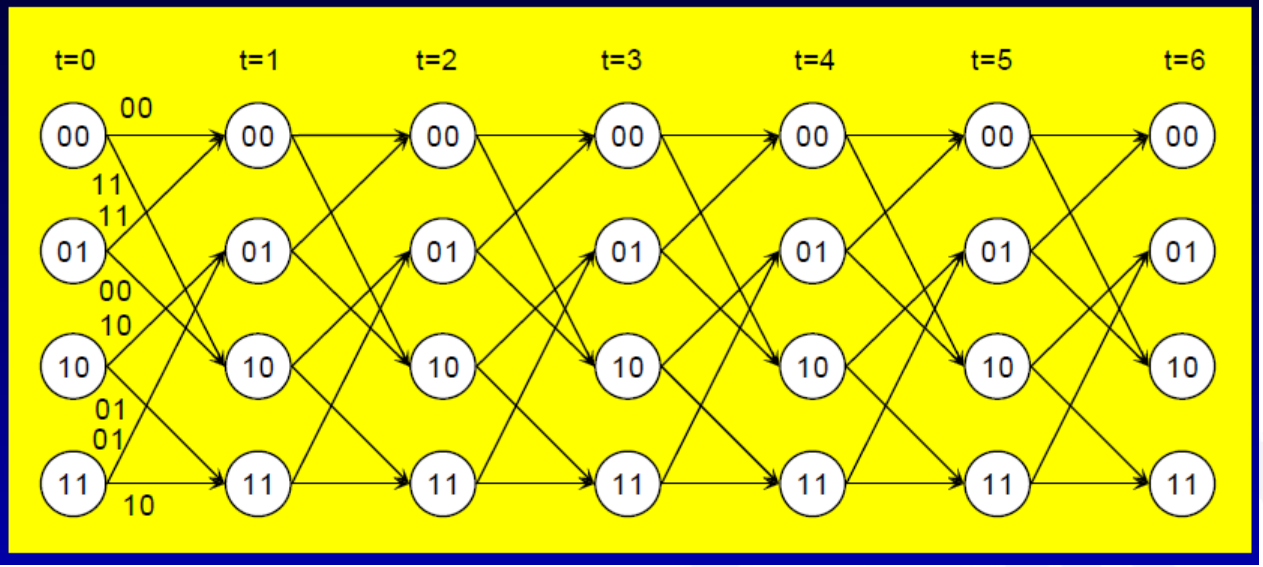
An FIR system

- Represent the impulse response for each output in octal form
- $(111, 101)_2=(7,5)_8$
- NASA standard (2, 1,7) convolutional code $(171, 133)_8$



Trellis struct

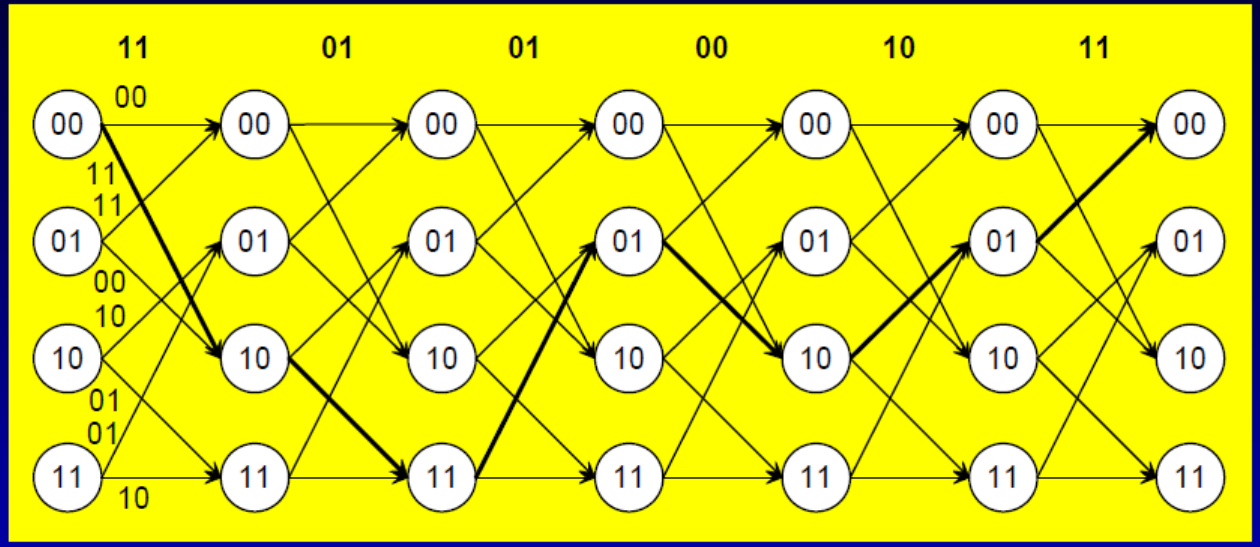
$$(s_0 s_1 \cdots s_{m-1}) \rightarrow (u s_0 s_1 \cdots s_{m-2})$$



- Expand the state di
  structure of convol
  and decoding.
- Shown is the trellis
- Note that state transitions (branches) from a state $s[i]$ to a (possibly different) state $s[i + 1]$, which are caused by the input bit $u[i] = 0$, are the upper branches.

Example of encoding

$\bar{u} = (110100)$



$\bar{v} = (11, 01, 01, 00, 10, 11)$

Why called “convolutional”

- The encoding is just the output of a discrete time linear system, which is equal to the convolution of input and impulse response of the system.

$$\bar{g}_j(D) = g_j[0] + g_j[1]D + g_j[2]D^2 + \cdots + g_j[m]D^m$$

$$v^{(j)}[i] = \sum_{l=0}^{m} u[i-l]g_j[l]$$

Semi-infinite generator matrix

- If you like, you can of course write the generator matrix of convolutional codes, as what we do in block codes.

$$G = \begin{pmatrix} g_0[0]g_1[0] & \cdots & g_0[m]g_1[m] & & \\ & g_0[0]g_1[0] & \cdots & g_0[m]g_1[m] & \\ & & g_0[0]g_1[0] & \cdots & g_0[m]g_1[m] \\ & & & \ddots & \ddots & \ddots \end{pmatrix}$$

$$G = \begin{pmatrix} 11 & 10 & 11 & & & \\ & 11 & 10 & 11 & & \\ & & 11 & 10 & 11 & \\ & & & 11 & 10 & 11 \\ & & & & \ddots & \ddots & \ddots \end{pmatrix}.$$

SUMMARY

Convolutional codes can be represented by encoder diagram, list of encoding, FSM diagram, trellis diagram, generator sequences, and generator matrix.

Trellis diagram is useful to understand the process of encoding and decoding.

8.4.2 VITERBI ALGORITHM

Maximum likelihood decoding

- For hard decision decoding, ML decoding is equivalent to minimum Hamming distance decoding.
- In the trellis , find a path which is closest to the received sequence in the sense of Hamming distance.
- But there are too many candidates!
- The number of candidates increase exponentially with time going.
- What shall we do?

Key observation of Viterbi (1967)

- With i being the time, most likely paths per state (the ones closest to the received sequence) will eventually coincide at some previous time $i - l$.
- l>5*memory length
- Decoding depth $L>l$

Viterbi AJ (April 1967). "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm". IEEE Transactions on Information Theory. 13 (2): 260–269.

Decoding steps (BMG)

$$\bar{v}[i] = (v_0[i]v_1[i]\dots v_{n-1}[i]) \Rightarrow \bar{r}[i] = (r_0[i]r_1[i]\dots r_{n-1}[i])$$

Initialization

Set $i = 0$. Set metrics and paths

$$M(S_0^{(k)}) = 0, \qquad \bar{y}_0^{(k)} = () \quad \text{(empty)}.$$

1. **Branch metric computation**

   At stage i, compute the partial *branch metrics*

   $$BM_i^{(b)} = d_H(\bar{r}[i], \bar{v}[i]),$$

   $b \triangleq \sum_{\ell=0}^{n-1} v_\ell[i]2^{n-1-\ell}$, associated with the n outputs $\bar{v}[i]$ of every branch (or state transition) and the n received bits $\bar{r}[i]$.

Decoding steps (ACS)

2. **Add, compare and select (ACS)**

For each state $S_i^{(k)}$, $k = 0, 1, \ldots, 2^m - 1$, and corresponding pair of incoming branches from two precursor states $S_{i-1}^{(k_1)}$ and $S_{i-1}^{(k_2)}$, the algorithm compares the extended branch metrics $M(S_{i-1}^{(k_1)}) + BM_i^{(b_1)}$ and $M(S_{i-1}^{(k_2)}) + BM_i^{(b_2)}$, where

$$b_j = \sum_{\ell=0}^{n-1} v_\ell[i] 2^{n-1-\ell} i = 1, 2,$$

and selects a *winning branch*, giving the smallest path metric, and updates the metric,

$$M(S_i^{(k)}) = \min\{M(S_{i-1}^{(k_1)}) + BM_i^{(b_1)}, M(S_{i-1}^{(k_2)}) + BM_i^{(b_2)}\}.$$

Decoding steps (survival paths)

3. **Path memory update**

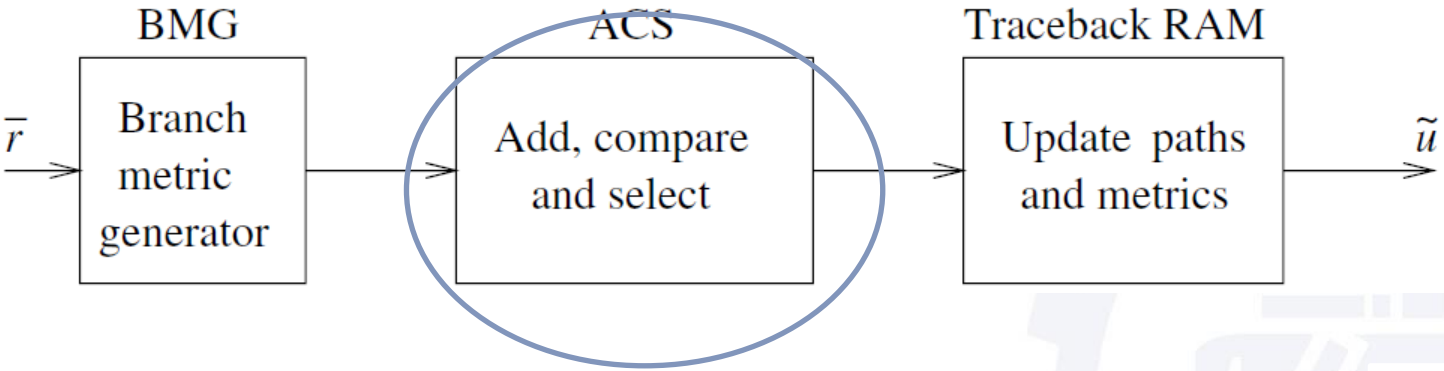
   For each state $S_i^{(k)}$, $k = 0, 1, \ldots, 2^m - 1$, update the *survivor paths* $\bar{y}^{(k)}$ as follows, with the output of the winning branch $\bar{v}_{k_j}$, $j \in \{1, 2\}$,

$$\bar{y}_i^{(k)} = (\bar{y}_{i-1}^{(k_j)}, \bar{v}_{k_j}).$$

4. **Decode symbols**

   If $i > L$, then output as the estimated coded sequence $\bar{y}_{i-L}^{(k')}$, where k' is the index of the state $S^{(k')}$ with the smallest metric. Set $i = i + 1$ and go to decoding step 1.

Block diagram of decoding



Decoding steps (survival paths)

3. **Path memory update**

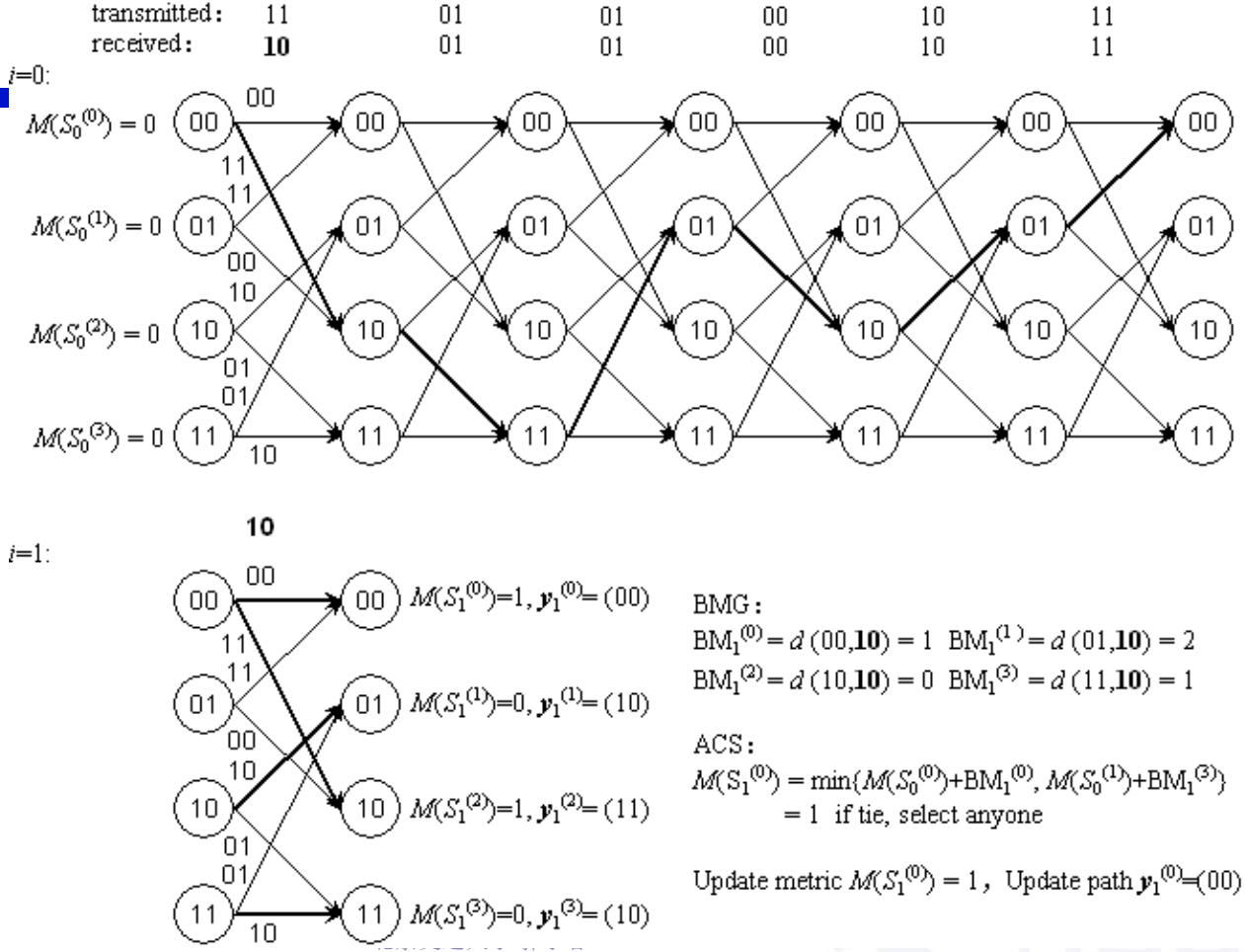
   For each state $S_i^{(k)}$, $k = 0, 1, \ldots, 2^m - 1$, update the *survivor paths* $\bar{y}^{(k)}$ as follows, with the output of the winning branch $\bar{v}_{k_j}$, $j \in \{1, 2\}$,

$$\bar{y}_i^{(k)} = (\bar{y}_{i-1}^{(k_j)}, \bar{v}_{k_j}).$$

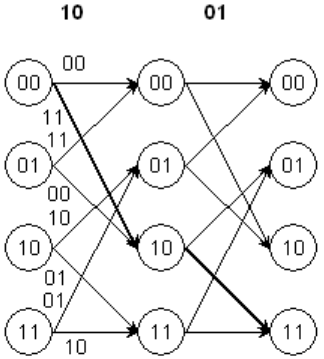
4. **Decode symbols**

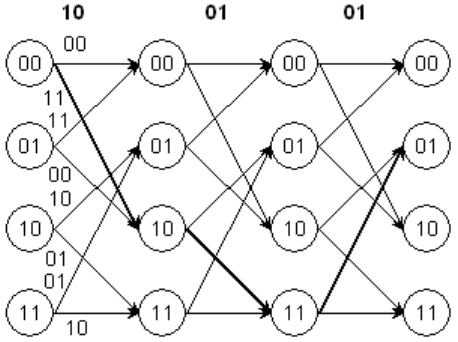
   If $i > L$, then output as the estimated coded sequence $\bar{y}_{i-L}^{(k')}$, where k' is the index of the state $S^{(k')}$ with the smallest metric. Set $i = i + 1$ and go to decoding step 1.

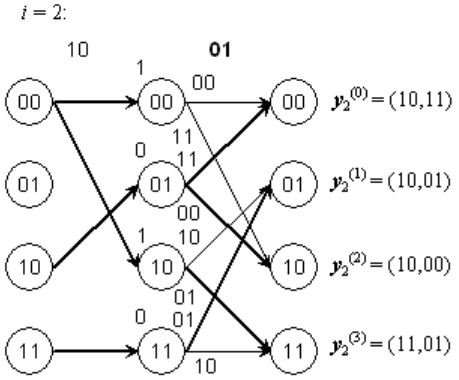
Example

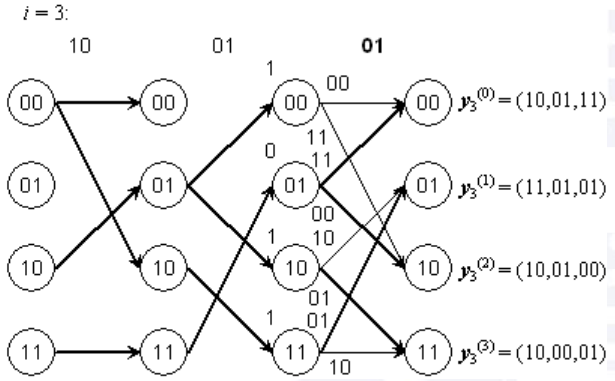

transmitted: 11 01 01 00 10 11
received: 10 01 01 00 10 11
i=0:
$M(S_0^{(0)}) = 0$
$M(S_0^{(1)}) = 0$
$M(S_0^{(2)}) = 0$
$M(S_0^{(3)}) = 0$
00
01
10
11
i=1:
$M(S_1^{(0)})=1, \boldsymbol{y}_1^{(0)}=(00)$
$M(S_1^{(1)})=0, \boldsymbol{y}_1^{(1)}=(10)$
$M(S_1^{(2)})=1, \boldsymbol{y}_1^{(2)}=(11)$
$M(S_1^{(3)})=0, \boldsymbol{y}_1^{(3)}=(10)$
BMG:
$BM_1^{(0)} = d(00,\mathbf{10}) = 1$ $BM_1^{(1)} = d(01,\mathbf{10}) = 2$
$BM_1^{(2)} = d(10,\mathbf{10}) = 0$ $BM_1^{(3)} = d(11,\mathbf{10}) = 1$
ACS:
$M(S_1^{(0)}) = \min\{M(S_0^{(0)})+BM_1^{(0)}, M(S_0^{(1)})+BM_1^{(3)}\}$
$= 1$ if tie, select anyone
Update metric $M(S_1^{(0)}) = 1$, Update path $\boldsymbol{y}_1^{(0)}=(00)$

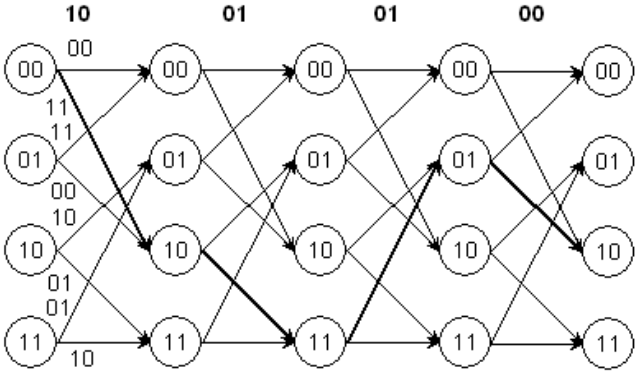
Example

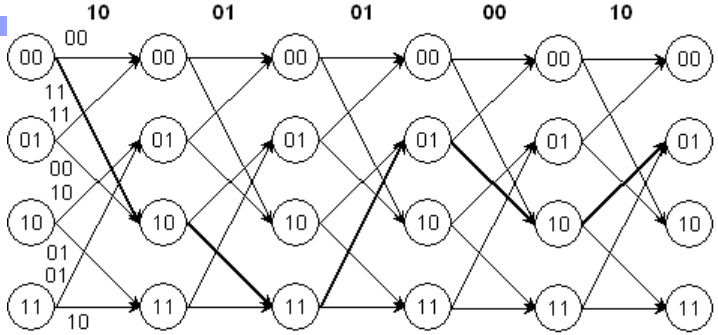

10
01
00
11
11
00
10
01
01
10
00
01
10
11
00
01
10
11


10
01
01
00
11
11
00
10
01
01
10
00
01
10
11
00
01
10
11

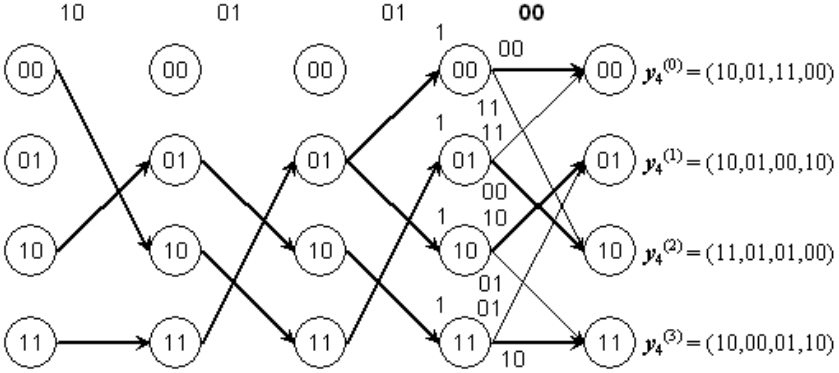

i = 2:
10
01
1
0
1
0
00
11
11
00
10
01
01
10
00
01
10
11
$y_2^{(0)} = (10,11)$
$y_2^{(1)} = (10,01)$
$y_2^{(2)} = (10,00)$
$y_2^{(3)} = (11,01)$


i = 3:
10
01
01
1
0
1
1
00
11
11
00
10
01
01
10
00
01
10
11
$y_3^{(0)} = (10,01,11)$
$y_3^{(1)} = (11,01,01)$
$y_3^{(2)} = (10,01,00)$
$y_3^{(3)} = (10,00,01)$

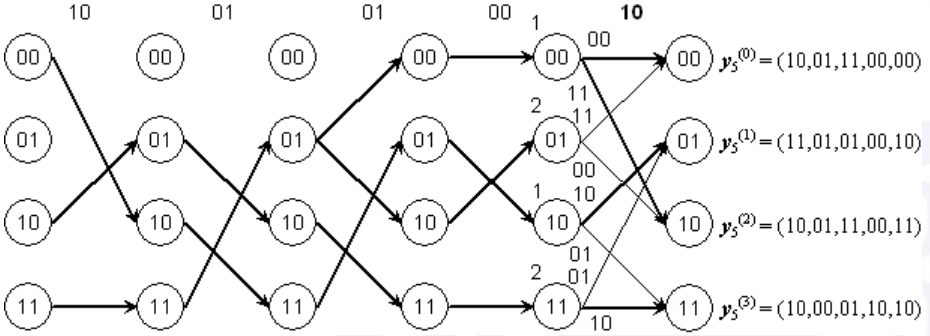

10
01
01
00
00
00
00
00
00
11
11
00
10
01
01
10


10
01
01
00
10
00
00
00
00
00
00
11
11
00
10
01
01
10

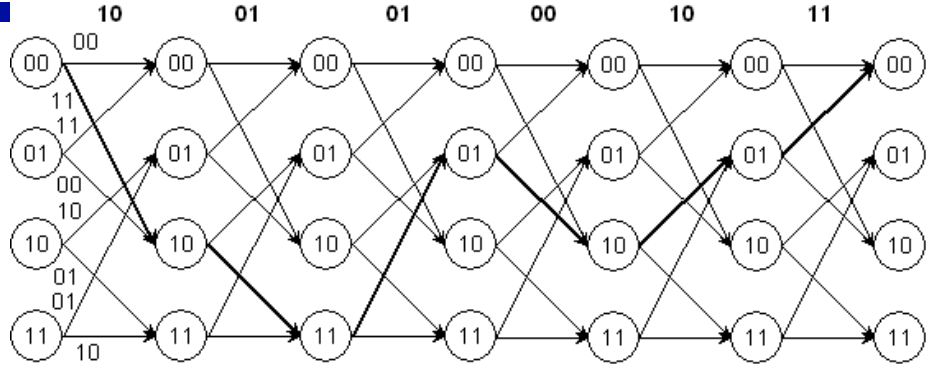
$i = 4$:


10
01
01
00
1
00
1
11
1
00
1
01
01
10
$y_4^{(0)} = (10,01,11,00)$
$y_4^{(1)} = (10,01,00,10)$
$y_4^{(2)} = (11,01,01,00)$
$y_4^{(3)} = (10,00,01,10)$

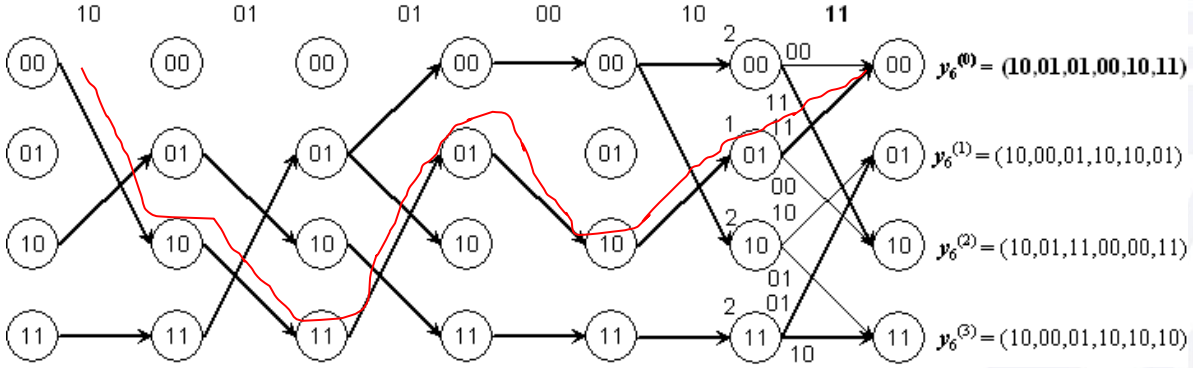
$i = 5$:


10
01
01
00
10
1
00
2
11
1
00
2
01
10
$y_5^{(0)} = (10,01,11,00,00)$
$y_5^{(1)} = (11,01,01,00,10)$
$y_5^{(2)} = (10,01,11,00,11)$
$y_5^{(3)} = (10,00,01,10,10)$

Example


10 01 01 00 10 11
00
01
10
11

$i = 6$:


10 01 01 00 10 11
00
01
10
11
$y_6^{(0)} = (10,01,01,00,10,11)$
$y_6^{(1)} = (10,00,01,10,10,01)$
$y_6^{(2)} = (10,01,11,00,00,11)$
$y_6^{(3)} = (10,00,01,10,10,10)$

SUMMARY

Survival paths will coincide finally.

Viterbi algorithm is actually a dynamic schedule algorithm.

ACS is the essential in Viterbi algorithm.